

Wiring LLM Hypotheses into PySR: the `extra_*_operators / seed_expressions` Mechanism

Hybrid-mode fix in `symbolic_engine.py`

1 Summary

`HybridDiscoverySystem`'s `llm_mode="hybrid"` path (`_discover_hybrid()`) and `llm_mode="seed"` path (`_discover_with_llm_seed()`) both computed an LLM-derived signal — `_use_llm_seed/llm_config` — and then discarded it before calling `super().discover()`. The subsequent PySR run was, in every case where refinement was supposed to add value, indistinguishable from a cold run with `llm_mode="none"`. This report explains the replacement mechanism: what it wires up, why it stops short of a true population warm-start, and how it degrades safely.

2 Why the obvious fix does not work

The natural first instinct is to seed PySR's initial population directly with the LLM's candidate expression, e.g. `guesses=[best_hyp.equation]`. We checked this against the installed `pysr` package (1.5.10) directly:

Listing 1: `PySRRegressor.__init__` has no working `guesses` parameter

```
1 >>> import inspect
2 >>> from pysr import PySRRegressor
3 >>> "guesses" in set(inspect.signature(PySRRegressor.__init__).parameters)
4 False
```

This is a known, currently-open upstream issue ([MilesCranmer/PySR #1057](#), “guesses parameter missing in `PySRRegressor`”): the parameter is referenced in some PySR documentation but is not accepted by the pip release actually installed. Critically, `PySRRegressor.__init__` is *not* a permissive `**kwargs` sink — unrecognized keywords raise `TypeError` rather than being silently ignored:

Listing 2: Unknown kwargs fail loudly, they are not swallowed

```
1 else:
2     suggested_keywords = _suggest_keywords(PySRRegressor, k)
3     err_msg = f"`{k}` is not a valid keyword argument for PySRRegressor."
4     ...
5     raise TypeError(err_msg)
```

So naively adding `guesses=...` would not have reproduced the original silent-no-op bug — it would have crashed every hybrid-mode run outright. Any fix has to either (a) use a parameter that is verified to exist, or (b) probe for the desired parameter at runtime and degrade gracefully if it is absent.

3 What is real: operator-set biasing

`_extract_operators_from_equation()` already parses an equation string and returns exactly:

```

1 {
2     "binary_operators": ["*", "/", ...],
3     "unary_operators": ["exp", "sin", ...],
4 }

```

`unary_operators` and `binary_operators` are real, verified `PySRRegressor.__init__` parameters. That is the legitimate wiring point used here: instead of constraining PySR to *only* the LLM’s operators (which would shrink the search space and could make results worse than a cold run), the fix takes the **union** of the LLM’s operators with everything the base `discover()` already assembles (domain heuristics, data-driven exp/log detection, transcendental-composition injection, etc.).

3.1 New hook in the base `discover()`

Two new keyword arguments are consumed — and popped out of `kwargs` so they never reach `PySRRegressor` directly — immediately after the existing domain/data-driven operator injection and before `pysr_kwargs` is finalized:

Listing 3: `extra_unary_operators / extra_binary_operators` hook (base `discover()`)

```

1 _extra_unary = kwargs.pop("extra_unary_operators", None) or []
2 _extra_binary = kwargs.pop("extra_binary_operators", None) or []
3 _seed_expressions = kwargs.pop("seed_expressions", None) or []
4 merged_binary_operators = list(self.config.binary_operators)
5
6 if _extra_unary or _extra_binary:
7     _added_unary = [op for op in _extra_unary if op not in active_unary]
8     active_unary.extend(_added_unary)
9     _added_binary = [op for op in _extra_binary
10                     if op not in merged_binary_operators]
11     merged_binary_operators.extend(_added_binary)
12     if _added_unary or _added_binary:
13         print(f"    [LLM-OPS] injected unary={_added_unary} "
14               f"binary={_added_binary}", flush=True)

```

`binary_operators` in the final `pysr_kwargs` dict is changed from the fixed `self.config.binary_operators` to the merged list:

```

1 pysr_kwargs = dict(
2     ...
3     binary_operators=merged_binary_operators,
4     unary_operators=active_unary,
5     ...
6 )

```

Because `active_unary` is extended *in place* before `pysr_kwargs` is built, the LLM’s operators automatically compose with every other operator-injection rule already in the method (trig-domain auto-injection, exp/log heuristics, transcendental compositions) rather than overwriting them.

3.2 Opportunistic true warm-start: `seed_expressions` → `guesses`

`seed_expressions` is also probed for, independently, right before construction:

Listing 4: Runtime probe for guesses, with safe fallback

```

1 _guesses_requested = False
2 if _seed_expressions:
3     _pysr_guess_params = set(
4         inspect.signature(PySRRegressor.__init__).parameters
5     )
6     if "guesses" in _pysr_guess_params:
7         pysr_kwargs["guesses"] = list(_seed_expressions)
8         _guesses_requested = True
9     else:
10        print("    [PySR] guesses param not available in this pysr "
11              "version -- falling back to operator biasing only",
12              flush=True)

```

If a future pysr release ships guesses with a different accepted format than a flat list of expression strings, construction may still fail. That failure is caught and retried *once* with guesses removed, rather than aborting the run:

Listing 5: Retry-without-guesses on construction failure

```

1 try:
2     self.model = PySRRegressor(**pysr_kwargs)
3 except Exception as _init_exc:
4     ...
5     if _guesses_requested and "guesses" in pysr_kwargs:
6         print("    [PySR] retrying construction without `guesses` "
7               "(operator-biasing ... is still in effect)", flush=True)
8         pysr_kwargs.pop("guesses", None)
9         self.model = PySRRegressor(**pysr_kwargs)
10    else:
11        raise

```

Against the currently installed pysr==1.5.10, the probe finds no guesses parameter, so this path is inert today and every run falls through to operator biasing only — verified with the installed package, not assumed.

4 The gate: `_use_llm_seed` in `_discover_hybrid()`

The R^2 threshold that decides whether the LLM’s prior is trustworthy is unchanged, but it now actually gates something:

Listing 6: `_discover_hybrid()`: the gate now has teeth

```

1 _use_llm_seed = True
2 _llm_extra_ops = None
3 if best_hyp.r2_score is None or best_hyp.r2_score < 0.5:
4     print(f"    LLM R2={best_hyp.r2_score:.3f} < 0.5 -- discarding LLM "
5           f"prior, running pure PySR", flush=True)
6     _use_llm_seed = False
7 else:
8     _llm_extra_ops = self._extract_operators_from_equation(best_hyp.equation)
9     kwargs.setdefault("extra_unary_operators",
10                       _llm_extra_ops["unary_operators"])
11    kwargs.setdefault("extra_binary_operators",
12                      _llm_extra_ops["binary_operators"])
13    kwargs.setdefault("seed_expressions", [best_hyp.equation])
14    print(f"    -> Seeding PySR operator set from LLM candidate: "
15          f"{_llm_extra_ops}", flush=True)
16

```

```

17 result = super().discover(X, y, variable_names, equation_name,
18                           random_state, auto_sanitise=auto_sanitise,
19                           **kwargs)

```

A weak LLM guess ($R^2 < 0.5$) never reaches either the operator-biasing hook or the opportunistic guesses attempt — it cannot narrow the operator search space, and it cannot be planted into a future population warm-start. This matters more for warm-start than for operator biasing: a bad operator hint just widens/narrows a menu PySR still searches freely, whereas a bad seed expression directly competes in generation 0 and can drag early tournament selection toward a wrong region of expression-space.

The existing compare-and-select logic at the end of `_discover_hybrid()` — run PySR, compare its R^2 against the LLM’s, keep whichever is better — was already correct and is unchanged. What changes is that when the LLM’s prior is used, it is used *before* the comparison, not only as a fallback candidate at the end.

4.1 Result transparency

Both `_discover_with_llm_seed()` and `_discover_hybrid()` now surface what actually happened, instead of leaving downstream benchmark code to infer it from the `llm_mode` string alone:

```

1 result["llm_suggested_operators"] = llm_config      # seed mode
2 result["llm_seed_used"]           = _use_llm_seed   # hybrid mode
3 result["llm_suggested_operators"] = _llm_extra_ops  # hybrid mode

```

5 Behavior by LLM R^2 range

LLM R^2	PySR called?	Interaction
> 0.95	No	LLM answer returned directly; PySR skipped entirely (unchanged).
$0.5\text{--}0.95$	Yes	New: LLM’s operators unioned into search space; guesses attempted if supported; result compared against LLM answer, better one returned.
< 0.5	Yes	PySR runs with its normal defaults, uninformed by the LLM. LLM answer still kept as a fallback candidate for the final comparison (unchanged).

6 Worked example

Given an LLM hypothesis `best_hyp.equation = "exp(x0)/x1"` with $R^2 = 0.71$ (inside the collaboration band):

Listing 7: What `_extract_operators_from_equation` returns for this input

```

1 >>> engine._extract_operators_from_equation("exp(x0)/x1")
2 {'binary_operators': ['/'], 'unary_operators': ['exp']}

```

`_discover_hybrid()` then calls the base `discover()` with:

```

1 super().discover(
2     X, y, variable_names, equation_name, random_state,
3     auto_sanitise=True,
4     extra_unary_operators=['exp'],
5     extra_binary_operators=['/'],

```

```

6     seed_expressions=['exp(x0)/x1'],
7 )

```

Inside the base method, suppose the config default is `unary_operators=["sqrt"]`, `binary_operators=["+", "-", "*", "/", "exp"]`, and the domain is "general" (no auto-trig/exp-log heuristics fire). The merge behaves as:

Listing 8: Resulting operator sets passed to PySRRegressor

```

1 active_unary          = ['sqrt', 'exp']          # 'exp' added, no duplicate
2 merged_binary_operators = ['+', '-', '*', '/']    # '/' already present, no-op

```

guesses is attempted (`pysr_kwargs["guesses"] = ["exp(x0)/x1"]`) but, against `pysr==1.5.10`, the probe reports guesses is unavailable, so that key is never added and construction proceeds with the merged operator sets only. The trace records "guesses=unavailable" so this is visible in the returned diagnostics rather than silent.

6.1 Contrast with the previous behavior

Listing 9: Before: computed, printed, discarded

```

1 _use_llm_seed = True
2 if best_hyp.r2_score is None or best_hyp.r2_score < 0.5:
3     _use_llm_seed = False
4 # _use_llm_seed is never read again.
5
6 result = super().discover(X, y, variable_names, equation_name,
7                           random_state, auto_sanitize=auto_sanitize,
8                           **kwargs)
9 # Identical to a cold run regardless of _use_llm_seed's value.

```

For the same $R^2 = 0.71$ example, the pre-fix Phase 2 call would have been

```

1 super().discover(X, y, variable_names, equation_name, random_state,
2                 auto_sanitize=True) # no LLM information reaches PySR

```

i.e. `unary_operators=['sqrt']`, `binary_operators=['+', '-', '*', '/']` — `exp` never enters the search space, so PySR cannot express the LLM's hypothesis at all regardless of how much search budget it is given.

7 Design notes

- **Union, not replacement.** Overwriting `unary_operators` with only the LLM's operators would shrink the search space and could make hybrid mode worse than `llm_mode="none"`. Unioning preserves every operator the base method would have included anyway.
- **Two independent probes, not one assumption.** The `define_operators` vs. embedded-body kwarg-name probe already existed in this file for a different PySR version-compatibility issue; the `guesses` probe follows the same pattern rather than hardcoding a parameter name that is verified *not* to work in the pinned release.
- **Failure of the opportunistic path never sinks the run.** `guesses` is popped and construction retried once on failure; only a genuine, unrelated construction failure still propagates.

- **The gate protects the stronger mechanism most.** Operator biasing is a soft constraint — PySR still searches freely within the (widened) menu. A literal population seed is a hard anchor that can dominate early tournament selection. Restricting `seed_expressions` to the $R^2 \geq 0.5$ branch means the riskier mechanism is the one most protected by the threshold.
- **`_discover_with_fallback()` is untouched.** It runs PySR first and only separately evaluates the LLM hypothesis if PySR underperforms — a legitimate compare-and-select pattern with nothing to inject, so it was not part of this bug.